

Data-Driven Design & Analysis of Structures & Materials (3dasm)

Instructor: Miguel A. Bessa

Brown University

Homework 4

Deliver a **ZIP folder with two files (PDF and Jupyter notebook)** containing the answers to the questions of this assignment. **UPLOAD to CANVAS in the Assignments section (Homework 4) by the due date.**

Due date: until 11:59pm of day announced in [course home page](#).

This Homework aims to help you reason about Linear Regression, using an AI coding agent to write and run the code. This is a homework containing questions to be solved with **pen and paper** as well as with **code**: the notebook you submit contains your prompts, the code the agents wrote, their results and your answers.

How to work. Start from the notebook `Assignments/3dasm_Homework4.ipynb`: copy it to `Assignments/your_Assignments/HW4/your_Homework4.ipynb` and work there. As in the lectures, an AI coding agent writes and runs the code, and the notebook is the source of truth: you write the agent's instructions file (`CLAUDE.md`) in a notebook cell, each prompt you give the agent is a notebook cell, and the cells that we provide load each agent's `results.json` from its task folder.

- **Model.** Use Sonnet or Haiku (`set_model("sonnet")` or `set_model("haiku")`); the course's helper does not accept other models.
- **Grading.** The TA runs your notebook **from scratch** (every agent runs again), with the model you declare in it. The regenerated numbers must be the same as the ones you submitted: if they are not, your instructions left something for the agent to guess. Your numbers are also compared with the solution: the homework fixes the dataset, the split and the models, so a precise notebook reproduces them exactly. Your predictions, checks and interpretations are graded as usual; base them on the numbers and figures your notebook stores, not on the agent's summary.
- **Prompts** are written in words. Short expressions that pin the data, the randomness or the method are fine (e.g. `random_state=123`); code that implements the task is not.
- Each exercise that says **(pen and paper)** should not be resolved with an AI agent (remember that you won't have access to AI in the midterm, so it's better for you not to use it for these questions)
- Each exercise that says **(with the agent)** is appropriate to resolve with an agent as we introduced in the lectures.
 - Note: **writing your own code instead of using the agent is allowed** for the exercises signalled as **(with the agent)**. Put your code in the same notebook cell (not in a separate file).
- **Before you submit**, delete the task folders and run the notebook from the start, so that its outputs come from one clean run (this is also what the TA will do when grading).
- **Submit one ZIP file** with your notebook (`.ipynb`), which the TA re-runs, and a PDF of that same notebook after your clean run. If you write the pen and paper answers by hand, add a scan of them as a third file. Do not include the task folders (they are not needed: the notebook is the source of truth).

Exercise 1 (pen and paper)

1. Conceptual questions.
 - 1.1. Explain from a Bayesian perspective the Linear Least Squares model and why do we use it to fit data with polynomial basis functions.
 - 1.2. What is a point estimate?

- 1.3. Explain what are training, validation and testing sets.
- 1.4. Define and explain the R^2 metric.
- 1.5. Define and explain what is the MSE metric.
- 1.6. Explain what is k -fold cross-validation.

Exercise 2 (pen and paper)

2. The matrix of basis functions.

- 2.1. Consider a two-dimensional input $\mathbf{x} = [x_1, x_2]^T$ and the quadratic basis functions vector in the order used by scikit-learn:

$$\phi(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]^T.$$

Write the matrix of basis functions Φ for the three training points $\mathbf{x}^{(1)} = [1, 2]^T$, $\mathbf{x}^{(2)} = [0, -1]^T$ and $\mathbf{x}^{(3)} = [2, 0]^T$. Give the dimensions of Φ , $\mathbf{w}$ and $\mathbf{y}$.

- 2.2. Is $\Phi^T \Phi$ invertible for the example of 2.1.? Justify your answer using the rank of Φ . More generally, what must be true about the number of training points N and the number of basis functions M for $\Phi^T \Phi$ to be invertible?
- 2.3. The maximum likelihood estimate of the noise variance is $\hat{\sigma}^2 = \text{RSS}/N$. Explain in your own words why it underestimates σ^2 , and write the unbiased alternative. When does the difference matter?
- 2.4. Write the plug-in posterior predictive distribution for a new input x^* . Which source of uncertainty does it ignore?

Exercise 3 (pen and paper)

Consider a linear regression model with two outputs, $\mathbf{y} = [y_1, y_2]^T$:

$$p(\mathbf{y}|\mathbf{x}, \mathbf{W}, \Sigma) = \mathcal{N}(\mathbf{y} | \mathbf{W}^T \phi(\mathbf{x}), \Sigma),$$

where $\phi(\mathbf{x})$ has M basis functions, $\mathbf{W}$ has one column per output, and Σ is a 2×2 covariance matrix, possibly with non-zero off-diagonal terms. The N training outputs are collected in the matrix $\mathbf{Y}$, one row per point.

3. Maximum likelihood estimate with two outputs.

- 3.1. Give the dimensions of $\mathbf{W}$, $\mathbf{Y}$ and Φ . Is Φ different from the single-output case?
- 3.2. Write the negative log likelihood and show that its minimum with respect to $\mathbf{W}$ is $\hat{\mathbf{W}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{Y}$, whatever Σ is. What does this mean for the columns of $\hat{\mathbf{W}}$? (Hint: write the condition that the gradient with respect to $\mathbf{W}$ is zero, and show that Σ^{-1} disappears when you multiply it on the right by Σ .)
- 3.3. Show that the maximum likelihood estimate of the covariance is $\hat{\Sigma} = \frac{1}{N} (\mathbf{Y} - \Phi \hat{\mathbf{W}})^T (\mathbf{Y} - \Phi \hat{\mathbf{W}})$.

Exercise 4 (with the agent)

Consider the function to be learned as $f(x) = x \sin(x)$ within the domain $x \in [0, 10]$.

4. Linear Least Squares with polynomial basis functions, for a dataset without noise.

- 4.1. Create a dataset without noise from $f(x)$ as follows (Homeworks 5 and 6 will use this exact file, so follow these instructions precisely):
 - Sample 40 values of x uniformly at random in $[0, 10]$, in a single draw from NumPy's default random number generator with seed 123 (`np.random.default_rng`). Note that "seed 123" alone is not enough: the older `np.random.seed(123)` gives different points.

- Sort the points in ascending order of x , and save them as a pandas dataframe with two columns, x and y , in a file called “HW4_noiseless_dataset.csv” (without the index) in the existing folder “your_data”.
- Split the sorted dataset into a training set with 80% of the points and a testing set with the remaining 20%, with the “train_test_split” function of scikit-learn and its “random_state” set to 123.

• **Important:** use the same training and testing sets for all the models of this homework.

- 4.2. Before comparing models, check that the computer solves the least squares problem correctly. With the entire training set (32 points), fit polynomials of every degree from 1 to 20 twice: once on the raw x (scikit-learn’s `PolynomialFeatures` and `LinearRegression`, with default arguments), and once with x first rescaled to the interval $[-1, 1]$ by scikit-learn’s `MinMaxScaler`, fitted on these training points. Plot the training MSE of both against the degree, with a logarithmic vertical axis. A polynomial of degree $d + 1$ can reproduce any polynomial of degree d (set its last weight to zero): what does this imply for the training MSE as the degree grows? Which of the two curves obeys it? (Your raw curve may differ from a classmate’s, depending on the version of scikit-learn: 5.3. explains why.) **From now on, rescale x in this way in every model of this exercise.**
- 4.3. **Before running anything else**, write down your predictions: when the models of the next question are trained with $N = 6, 11, 21$ and 32 points, which polynomial degree do you expect to predict the testing points best, and what do you expect each model to do outside the domain $x \in [0, 10]$? (You will not lose points for wrong predictions. You will lose points for not writing them.)
- 4.4. Train Linear Least Squares models, with x rescaled as in 4.2., but with the `MinMaxScaler` fitted on the training points of each model only, with polynomials of degree 1, 3, 5, 10 and 20, using four different training set sizes: the first 6, 11, 21 and 32 points of the training set of 4.1., in the order in which “train_test_split” returns them. For every model, calculate the MSE on its training points, and the R^2 and MSE on the testing set. Present them as suggested in Table 1, and show 4 figures (one per training set size) with the training points, the testing points, $f(x)$ and the polynomial approximations, for $x \in [-2, 12]$.

Table 1: Suggested table to report the testing R^2 (similar for the testing and the training MSE) in each cell. (NOTE: You do not need to present the values as in this table, as long as you show the same information.)

	degree	1	3	5	10	20
n_{train}						
6						
11						
21						
32						

- 4.5. Now consider the **entire training set** (32 points) and use 4-fold cross-validation, with the points shuffled before they are divided into folds (with seed 123), to calculate the **mean** and **standard deviation** of the R^2 and MSE metrics for the same five models as in 4.4. (the scaler is then fitted on the training folds only; the standard deviation is that of the 4 values, as computed by NumPy). Present the results as suggested in Table 2. **Explain the obtained mean and standard deviation.** Note that each fold has as many points as the testing set.

Table 2: Suggested table to report R^2 and MSE using cross-validation, where each cell contains their mean and standard deviation over the 4 folds.

	degree	1	3	5	10	20
metric						
MSE		mean $\pm$ std	mean $\pm$ std	mean $\pm$ std	mean $\pm$ std	mean $\pm$ std
R^2		mean $\pm$ std	mean $\pm$ std	mean $\pm$ std	mean $\pm$ std	mean $\pm$ std

- 4.6. **What can you conclude from these results?** Compare them with your predictions of 4.3., using the terms underfitting, overfitting, interpolation and extrapolation. In particular:

- For which training set sizes and degrees is the training MSE (almost) zero, and why?
- Where are the testing errors large, and why? (Hint: for each training set size, compare where the testing points are with where the training points are.)
- Is the testing R^2 a good guide here? Look at where the testing points are, and compare with the cross-validation of 4.5..
- Compare with the noisy example seen in class: where does overfitting come from in each case?

Exercise 5 (with the agent)

5. Check the agent's results against the derivation of the Linear Least Squares solution.
 - 5.1. For the model of degree 3 on the raw x (without rescaling), trained with the first 11 training points of 4.4., report the weights obtained by scikit-learn, and the weights $\hat{\mathbf{w}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$ computed explicitly with NumPy, with the inverse of the matrix (not with a least-squares solver). If computing the inverse fails, your code must record the error and continue; if it does not fail, that does not mean that its result is right. Do the two sets of weights agree? Quote the lines of the agent's code where Φ is built, and explain where the scikit-learn model stores the weight w_0 .
 - 5.2. Repeat 5.1. for the model of degree 20 trained with the first 6 training points. Do the two sets of weights agree? Plot both models on the same figure. In exact arithmetic, how many polynomials of degree 20 pass through 6 points (use your answer to 2.2.)? Do the two models pass through the 6 training points? Report the smallest and the largest entries of Φ , and explain why the computer struggles with this problem even though the math is simple. (Hint: a computer keeps about 16 significant digits; compare that with the ratio of the largest to the smallest entry of Φ .)
 - 5.3. Which of the infinitely many solutions does scikit-learn return in that case? Which directions does it treat as zero, and does this explain the raw curve of 4.2.? (Hint: read the documentation of `LinearRegression`, including its parameter `tol`, and of the function it uses to solve the problem.)

Exercise 6 (with the agent)

6. Linear regression with two outputs, for the model of Exercise 3.
 - 6.1. Create a dataset with two noisy outputs: with NumPy's default random number generator and seed 123, first draw 40 values of x uniformly at random in $[0, 10]$, and then 40 noise vectors $[\epsilon_1, \epsilon_2]^T$ with the generator's `multivariate_normal`, with zero mean and covariance $\Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$. The outputs are $y_1 = x \sin(x) + \epsilon_1$ and $y_2 = x \cos(x) + \epsilon_2$. Split the dataset into training (80%) and testing (20%) sets with "train_test_split" and its "random_state" set to 123. With polynomial basis functions of degree 5 and the training set, verify that three computations give the same weights: one scikit-learn model fitted to both outputs at once, two scikit-learn models fitted to one output each, and the expression of 3.2. Report $\hat{\Sigma}$ (the maximum likelihood estimate of 3.2., from the training residuals), and compare it with the true Σ .
 - 6.2. If Σ does not change the weights, what does knowing it give you? Give an example of a decision that depends on both outputs at the same time, for which the off-diagonal term of Σ matters.